UNIVERSIDAD COOPERATIVA DE COLOMBIA · SEMANA 10

Programación de Entornos Multimedia

Físicas reales con Cannon.js y check-list del proyecto final

Mg. Gabriel Guerrero Arellano



UNIVERSIDAD COOPERATIVA DE COLOMBIA

01

Parte 1: Recorderis general del curso

Lo que hemos trabajado

Parte 1 | Parte 2 | Parte 3

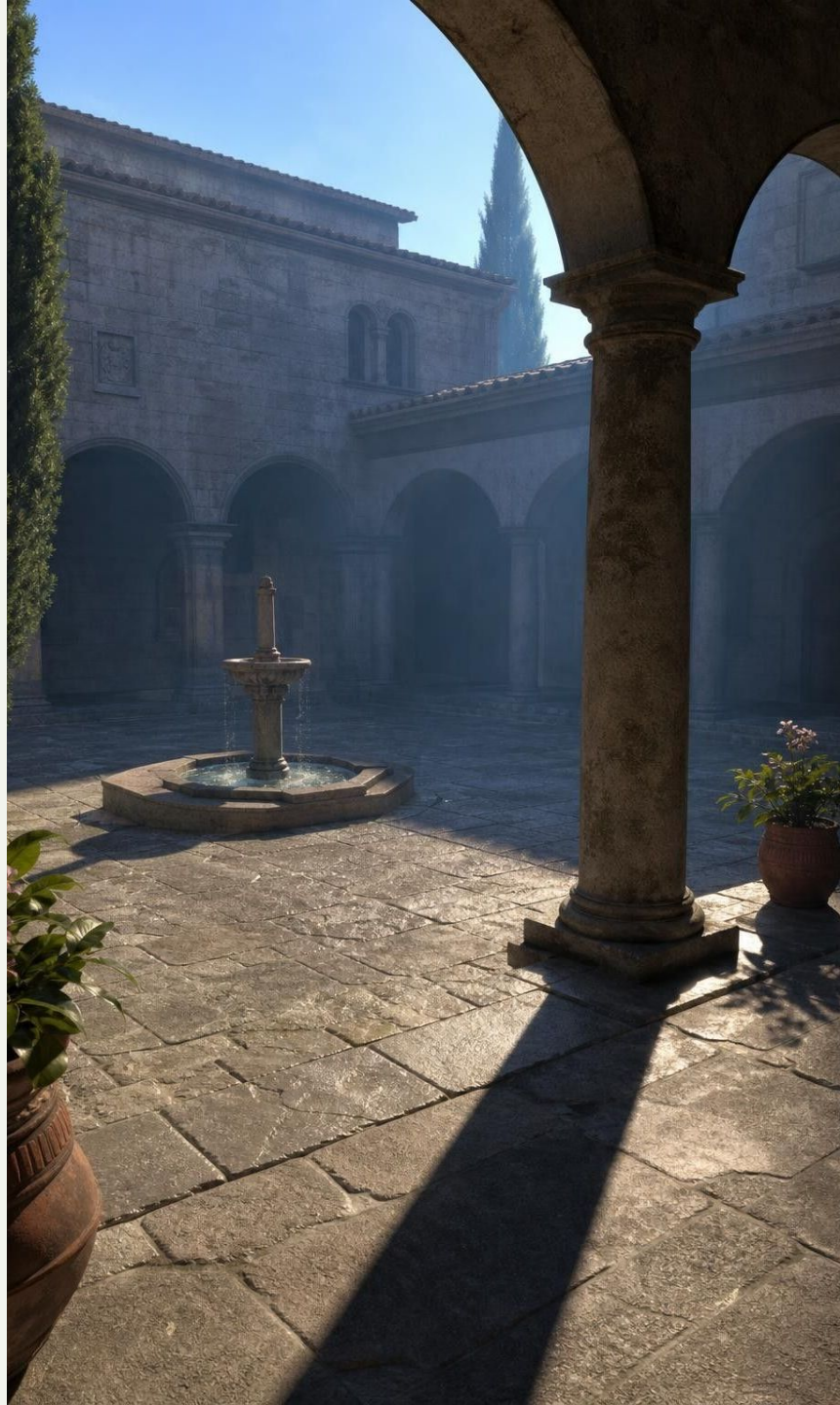
UNIVERSIDAD COOPERATIVA DE COLOMBIA · BLOQUE 1

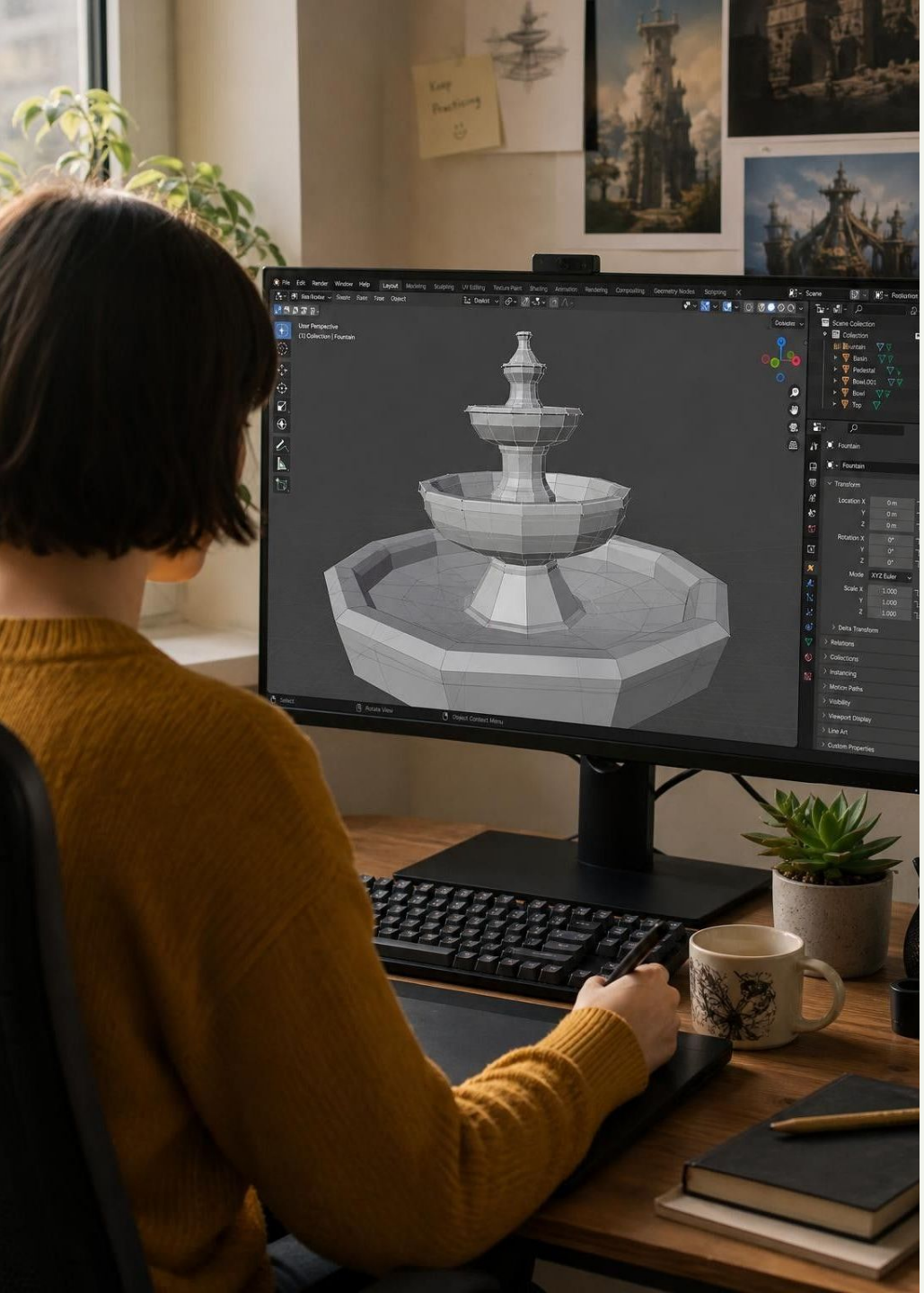
Fundamentos del renderizado 3D y WebGL

El lienzo y la matriz: configuración del `WebGLRenderer`, manejo del `canvas` HTML5 y eliminación de desbordamientos con CSS (`pointer-events: none` para el HUD).

La perspectiva del observador: `PerspectiveCamera` (FOV, Aspect Ratio, Near/Far planes) y navegación con `OrbitControls`.

Iluminación y atmósfera: `AmbientLight`, `DirectionalLight` con sombras y niebla de profundidad (`scene.fog = FogExp2`).





UNIVERSIDAD COOPERATIVA DE COLOMBIA · BLOQUE 2

Asset pipeline y modelado 3D

01

De Blender a WebGL

Modelado, optimización poligonal y exportación en formato `.glb` / `.gltf`. con la filosofia low poly para mantener el rendimiento en nuestro entorno web.

02

Carga asíncrona

`GLTFLoader` con funciones *callback* para no congelar la pestaña durante la descarga.

Audio 3D dual

Audio posicional

PositionalAudio

Atenuación sonora por distancia radial en la fuente de agua.

Audio global

THREE.Audio

Feedback instantáneo de eventos como el teletransporte.

Gestión de permisos

AudioContext

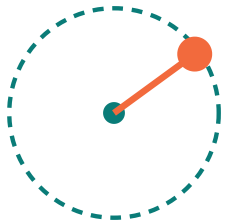
Desbloqueo tras la primera interacción del usuario a traves de un evento listener.

Detección de interacción (triggers)

Radial

Triggers radiales

distanceTo()

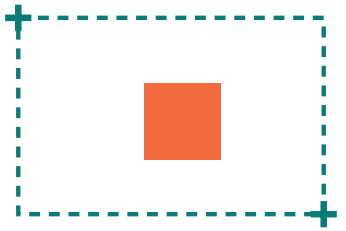


Vector3.distanceTo() para medir distancias a objetos circulares.

AABB

Triggers AABB

Box3()



THREE.Box3() con control de estado (*cooldown*) para zonas cuadradas y portales.

UNIVERSIDAD COOPERATIVA DE COLOMBIA · BLOQUE 3

Sistemas de partículas en GPU

Memoria

`BufferGeometry` con `Float32Array` para gestionar cientos de puntos en memoria continua de bajo nivel.

Simulación

Gravedad y reciclaje (*pooling*) dentro del bucle `animate()` mediante `needsUpdate = true`.





UNIVERSIDAD COOPERATIVA DE COLOMBIA · 02

Parte 2: El nuevo componente — físicas reales con Cannon.js

Three.js dibuja lo que se ve, pero **Cannon.js calcula lo que se siente.**

La arquitectura espejo: física y render desacoplados

CANNON.JS

Mundo físico `CANNON.World`

Existe en memoria paralela con su propia gravedad (Y = -9.81 m/s²).

Cuerpos rígidos `CANNON.Body`

Objetos invisibles con masa, forma (esferas, cajas, planos) y velocidad.



THREE.JS

Scene

Mallas visuales `THREE.Mesh`

Las figuras visibles de Three.js.

Sincronización: copia posición y rotación en cada frame de `animate()`. Cannon.js calcula la nueva posición física y se la copia a la malla visual.

1. Importación e inicialización del mundo físico

```
import * as CANNON from 'cannon-es';
// Crear el mundo físico
const world = new CANNON.World({
    gravity: new CANNON.Vec3(0, -9.81, 0) // Gravedad terrestre en Y
});
world.broadphase = new CANNON.NaiveBroadphase(); // Detección de colisiones simple
// Materiales físicos (Fricción y Restitución/Rebote)
const materialFisicoBase = new CANNON.Material('base');
const contactoMaterial = new CANNON.ContactMaterial(
    materialFisicoBase,
    materialFisicoBase,
    { friction: 0.3, restitution: 0.7 } // 0.7 = Rebote alto (tipo pelota)
);
world.addContactMaterial(contactoMaterial);
```

2. Creación del suelo físico (especular al suelo de Three.js)

```
// Suelo Físico en Cannon.js
const sueloShape = new CANNON.Plane();
const sueloBody = new CANNON.Body({ mass: 0, material: materialFisicoBase }); // masa 0 = objeto estático
sueloBody.addShape(sueloShape);
sueloBody.quaternion.setFromEuler(-Math.PI / 2, 0, 0); // Rotar para que quede horizontal
world.addBody(sueloBody);
```

3. Objeto dinámico interactivo (pelota cayendo)

```
// A. PARTE VISUAL (Three.js)
const pelotaGeo = new THREE.SphereGeometry(0.5, 32, 32);
const pelotaMat = new THREE.MeshStandardMaterial({ color: 0xff0055, roughness: 0.2 });
const pelotaMesh = new THREE.Mesh(pelotaGeo, pelotaMat);
scene.add(pelotaMesh);
// B. PARTE FÍSICA (Cannon.js)
const pelotaShape = new CANNON.Sphere(0.5);
const pelotaBody = new CANNON.Body({
    mass: 1, // Masa > 0 = Objeto afectado por la gravedad
    position: new CANNON.Vec3(0, 10, 0), // Nace a 10 metros de altura
    shape: pelotaShape,
    material: materialFisicoBase
});
world.addBody(pelotaBody);
```

4. Sincronización dentro del bucle animate()

```
const clock = new THREE.Clock();
let tiempoAnterior = 0;
function animate() {
    requestAnimationFrame(animate);
    const tiempoTranscurrido = clock.getElapsedTime();
    const deltaTime = tiempoTranscurrido - tiempoAnterior;
    tiempoAnterior = tiempoTranscurrido;
    // 1. Avanzar la simulación física (paso de tiempo fijo 60fps)
    world.step(1 / 60, deltaTime, 3);
    // 2. Sincronizar la malla de Three.js con el cuerpo físico de Cannon.js
    pelotaMesh.position.copy(pelotaBody.position);
    pelotaMesh.quaternion.copy(pelotaBody.quaternion);
    // ... (resto de animaciones de portal, partículas y triggers) ...
    renderer.render(scene, camera);
}
```

03

Parte 3: Check-list definitivo para la entrega del proyecto final

Lo que cada proyecto debe cumplir en las semanas de cierre.

Requisitos técnicos obligatorios (1–4)

#	Módulo	Requisito técnico
1	**Escenario 3D**	Escenario completo importado desde Blender (.glb) con iluminación y sombras configuradas.
2	**Navegación**	Control de cámara fluido mediante OrbitControls o PointerLockControls.
3	**Audio 3D**	Al menos UN (1) emisor PositionalAudio anclado a un objeto y UN (1) audio de evento global (THREE.Audio).
4	**Triggers**	Al menos DOS (2) triggers funcionales (1 Radial con distanceTo y 1 por Caja AABB con Box3).

Requisitos técnicos obligatorios (5–8)

#	Módulo	Requisito técnico
5	Partículas	Un sistema dinámico basado en BufferGeometry + Float32Array con física de simulación (agua, fuego, humo o lluvia).
6	Físicas Cannon.js o motor de físicas	Al menos UN (1) objeto dinámico (pelota, caja, obstáculo) con masa, gravedad y rebote sincronizado con Three.js.
7	Interfaz HUD	Panel flotante HTML/CSS sobre el canvas mostrando zona actual y coordenadas en tiempo real.
8	Optimización	Tasa de refresco fluida (objetivo: 60 FPS) sin instanciación de objetos dentro de animate().